```
#enable the sched_switch event and
$ lttng enable-event -k sched_switch
kernel event sched_switch created in channel channel0

#start tracing
$ lttng start

#start sleeping
$ sleep 1

#stop tracing
$ lttng stop
Waiting for data availability.
Tracing stopped for session osfy
```

So all the *sched_switch* commands between the start and stop of traces are traced, and the traces are written in */home/suchakra/lttng-traces/osfy-20131227-220359*. We can have a quick look at them using *Babeltrace*. Alternatively, the *lttng view* command calls *babeltrace* as the default viewer.

```
$ babeltrace /home/suchakra/lttng-traces/osfy-20131227-220359
```

This will list all the events with timing and other related context information like *prev_comm*, *next_comm*, *next_tid*, etc, per line. The problem is that there is an information overload for the user. In fact, we can do the following:

```
$ lttng view | wc -l
14520
```

Observe that LTTng recorded a total of 14520 *sched_switch* events in the short tracing duration, which is a lot to understand in one go. To see the events of interest (those related to the sleep command), take a look at the following code snippet:

```
$ lttng view | grep sleep
[22:14:45.118927309] (+0.004878641) isengard.localdomain
sched_switch: { cpu_id = 3 }, { prev_comm = "swapper/3",
prev_tid = 0, prev_prio = 20, prev_state = 0, next_comm =
"sleep", next_tid = 11766, next_prio = 20 }
[22:14:45.119069564] (+0.000000194) isengard.localdomain
sched_switch: { cpu_id = 3 }, { prev_comm = "sleep", prev_
tid = 11766, prev_prio = 20, prev_state = 64, next_comm =
"swapper/3", next_tid = 0, next_prio = 20 }
[22:14:45.147798113] (+0.000301434) isengard.localdomain
sched_switch: { cpu_id = 1 }, { prev_comm = "sleep", prev_
tid = 11801, prev_prio = 20, prev_state = 1, next_comm =
"swapper/1", next_tid = 0, next_prio = 20 }
.
.
.
```

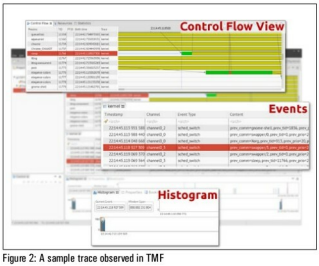

Figure 2: A sample trace observed in TMF

Just a cursory look at the above code can tell you that each line is a single *sched_switch* event recorded from the kernel. The timestamps are high precision as you can see from two consecutive events. The one in parenthesis is the'delta', i.e., the time between the previous event and the current one. The *cpu_id* tells the CPU for which the event was scheduled and various other context information is attached. All this is part of the CTF trace written.

After tracing is over, we can destroy the current tracing session as follows:

```
$ lttng destroy
```

Going further, we can use nice GUI tools such as Eclipse TMF for analysing the trace. Figure 2 shows what similar information would look like in TMF. You can see the timeline and control flow view, which is more intuitive. In the next article we will go into the details about userspace tracing with some real life examples and then move on to explore how to analyse a trace with TMF. Happy tracing! END

Resources

In the meantime, have a look at *http://lttng.org/documentation* and *http://www.youtube.com/user/lttng* for more information. Note that the videos are a bit old and some steps may vary.

Acknowledgements

Thanks to Simon Marchi and Francis Giraldeau for reviewing this article and to the 'tracing folks' at EfficiOS, Ericsson and École Polytechnique de Montréal.

By: Suchakrapani Sharma

The author is a PhD student at École Polytechnique de Montréal. He is currently doing research on dynamic tracing tools, and has varied interests—from performance analysis tools to embedded Linux and UX/graphics design. For more details, visit *http://suchakra.wordpress.com*